ROBÓTICA · SISTEMAS DE CONTROL

Control y seguimiento de trayectorias en un vehículo con geometría de Ackermann

Auto LEGO Technic 42160 (Audi RS e-tron GT) · trayectorias de Reeds-Shepp · MicroPython + Pybricks

Material de cátedra · Robótica y Sistemas de Control · Ing. Juan P. Pedroni · Mayo 2026

CONTENIDOS

Qué vamos a recorrer

- Introducción: el problema y la arquitectura del sistema
- Modelado del vehículo: Ackermann, modelo de bicicleta y dinámica del vehículo
- El sistema de control visto como un sistema
- Control PI: teoría e implementación
- Generación de las referencias en tiempo de
- Sensores: rumbo (IMU) y encoders increme
- Actuadores: los motores LEGO
- Generación de trayectorias de Reeds-Shepp
- Simulación: el prototipo en MATLAB y su im
- Implementación: código, dashboard y robus
- Validación, limitaciones y cierre



INTRODUCCIÓN

El problema: seguir una trayectoria

- Seguir una trayectoria es lograr que un vehículo real recorra un camino geométrico planificado, a pesar de errores de modelo, deslizamiento y juego mecánico.
- Aparecen dos preguntas separadas:
  - ¿Qué camino? — problema de planificación de trayectorias.
  - ¿Cómo recorrerlo? — problema de control realimentado.
- En este proyecto el camino lo da el planificador de Reeds-Shepp y el seguimiento lo resuelven dos lazos de control PI.
- El vehículo tiene geometría de Ackermann: no puede girar en el lugar ni desplazarse de costado (restricción no holónoma).

IDEA CLAVE

Planificar y controlar son dos problemas distintos

Primero se decide el camino (geometría). Después se lo sigue con realimentación. Mezclarlos lleva a sistemas frágiles; separarlos los hace entendibles y robustos.

INTRODUCCIÓN

Del concepto teórico al auto LEGO

- Punto de partida: el set comercial LEGO Technic 42160, el Audi RS e-tron GT, con su hub programable y sus motores.
- Se reemplazó el comportamiento original por código propio escrito en MicroPython sobre el entorno Pybricks.
- El auto genera y sigue trayectorias de Dubins y de Reeds-Shepp entre poses (x, y, rumbo) que define el usuario.
- Doble implementación, y eso es deliberado:
  - Prototipo y validación del algoritmo en MATLAB.
  - Port del mismo algoritmo a MicroPython para correr en el hub.
- Un dashboard web monitorea el ensayo en vivo y permite ajustar las ganancias.

IDEA CLAVE

Un banco de pruebas real, accesible y reproducible

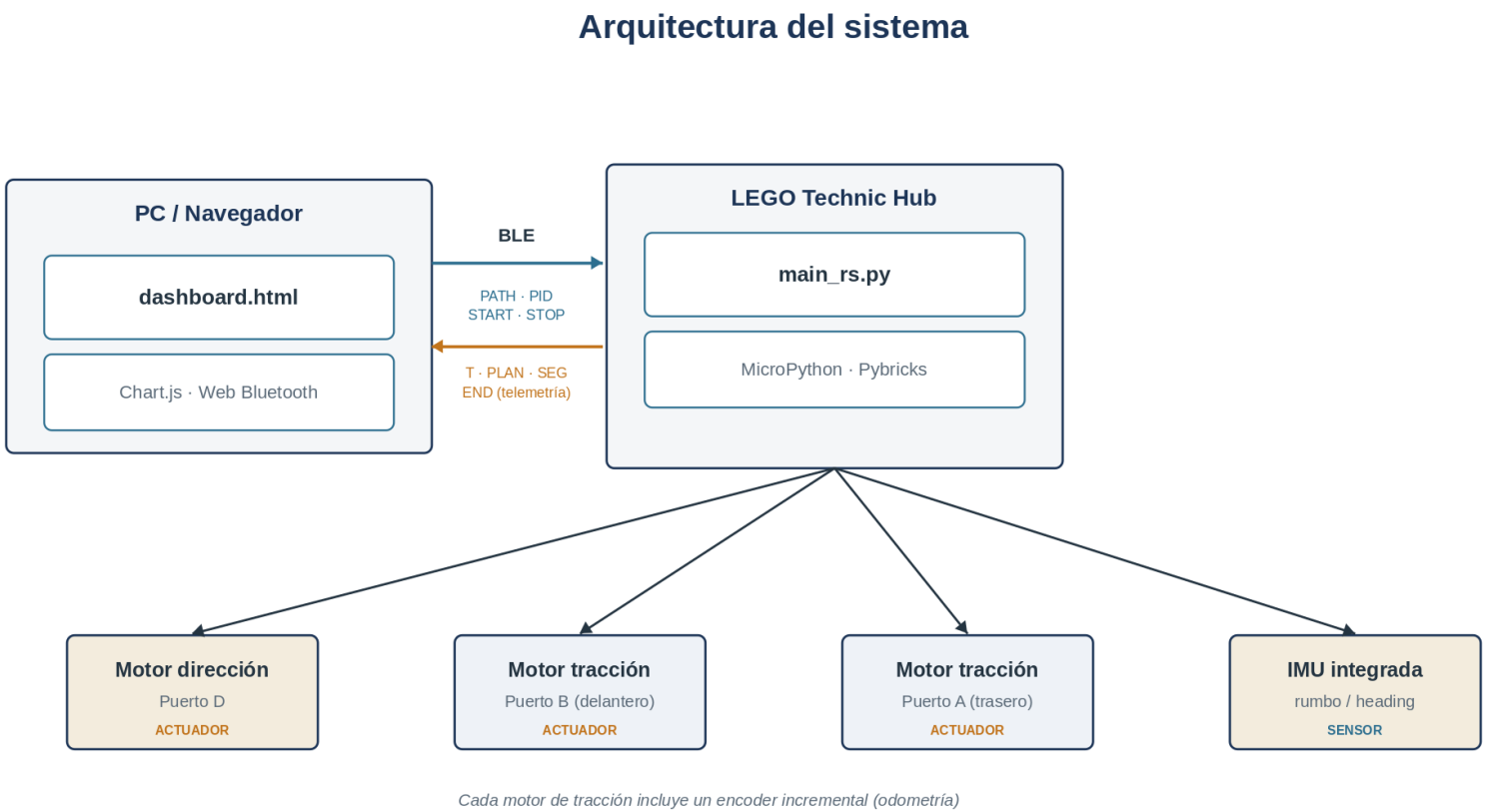
El LEGO permite tocar de verdad todos los conceptos de la cátedra: modelo, sensores, actuadores, control y planificación, sin el costo de un vehículo real.

El vehículo físico: parámetros

Valores definidos en vehicleConstants.py — son las constantes que cierran el modelo.

24 cm	20°	39 cm/s	80 cm
Batalla L	**Ángulo máx. ψ**	**Velocidad v**	**Radio mínimo**
distancia entre ejes delantero y trasero	giro máximo de las ruedas delanteras	velocidad lineal, constante durante el ensayo	radio de giro de diseño (r_turn_min)

Arquitectura del sistema



Dashboard en la PC ↔ enlace BLE ↔ hub LEGO ↔ motores y sensores.

MODELADO

¿Por qué geometría de Ackermann?

- En una curva, cada rueda recorre un círculo de radio distinto alrededor de un mismo centro: el centro instantáneo de rotación (ICR).
- Si las dos ruedas delanteras giraran el mismo ángulo, una de ellas arrastraría de costado: deslizamiento, desgaste y trayectoria impredecible.
- La geometría de Ackermann hace que la rueda interior gire más que la exterior, de modo que ambas rueden sin arrastre.
- Es la solución que usan los autos reales desde hace más de un siglo.
- Para el control nos interesa la consecuencia: existe un único ICR y, por lo tanto, un radio de giro bien definido para cada ángulo de dirección.

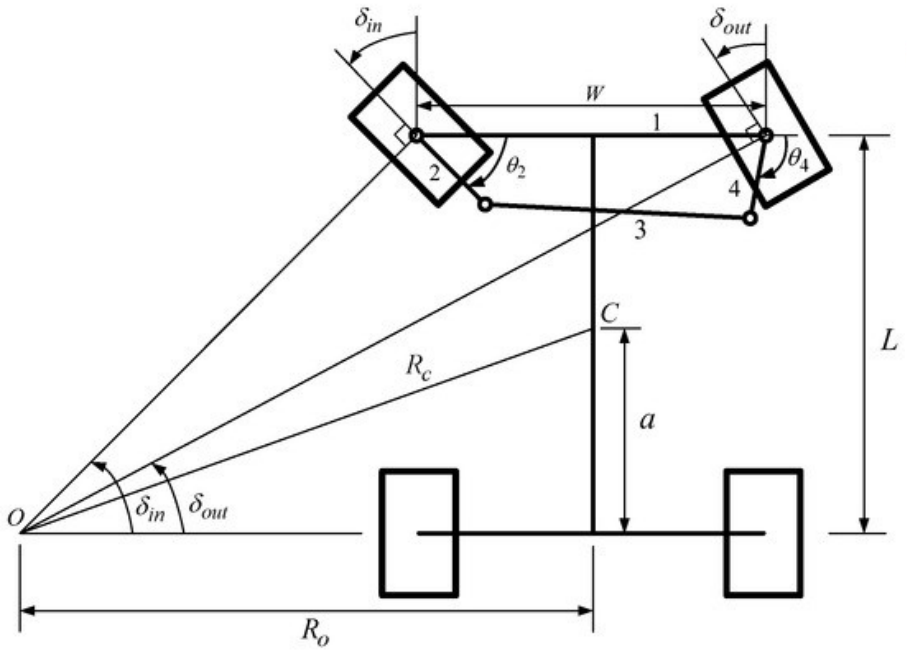
IDEA CLAVE

Rodadura pura: condición de un buen modelo

Si las ruedas no deslizan, el movimiento es puramente geométrico y predecible. Sobre esa hipótesis se construye todo el modelo cinemático.

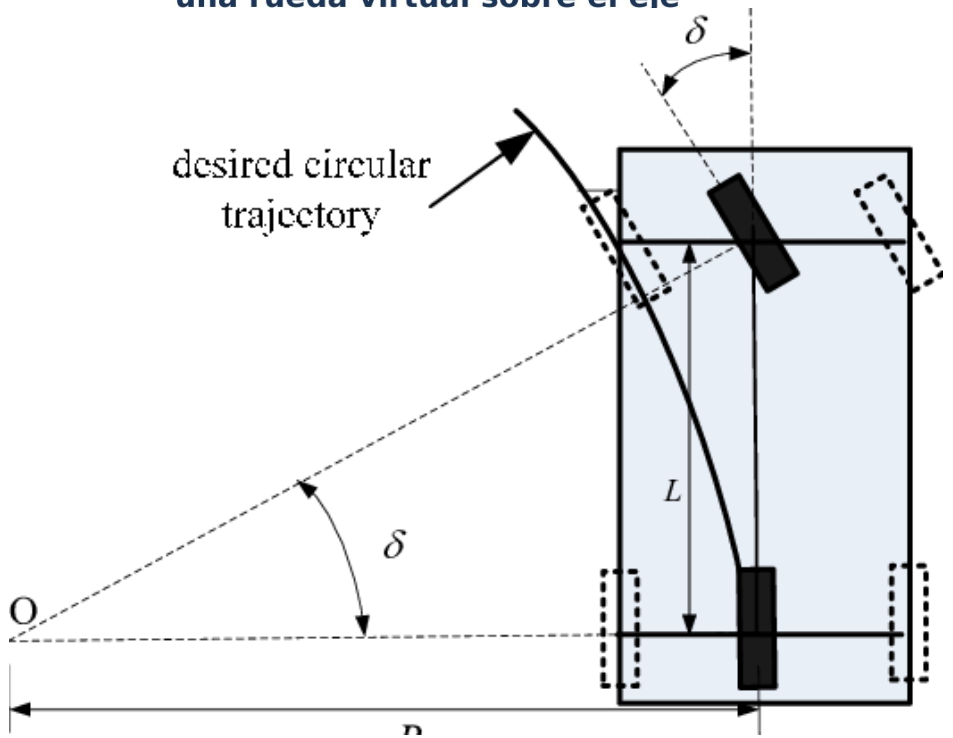
Geometría de dirección de Ackermann

Ackermann real
las dos ruedas giran ángulos distintos



$$\cot \delta_o - \cot \delta_i = t/L$$

Modelo de bicicleta equivalente
una rueda virtual sobre el eje



$$\tan \delta = L/R$$

Vista superior: ángulos interior y exterior, ICR y radio de giro.

MODELADO

Ecuaciones de la geometría de Ackermann

- **Condición de Ackermann:** $\cot \delta_o - \cot \delta_i = t / L$
- Relaciona el ángulo exterior δo, el interior δi, la trocha t y la batalla L.
- **Ángulo de dirección equivalente:** $\tan \delta = L / R$
- δ es el ángulo de una rueda virtual sobre el eje longitudinal; R es el radio del círculo que describe el eje trasero.
- Cuanto mayor el ángulo δ, menor el radio R: el vehículo gira más cerrado.
- El ángulo físico está limitado (ψmáx = 20°): de ahí sale un radio de giro mínimo, que es lo que el planificador no puede violar.

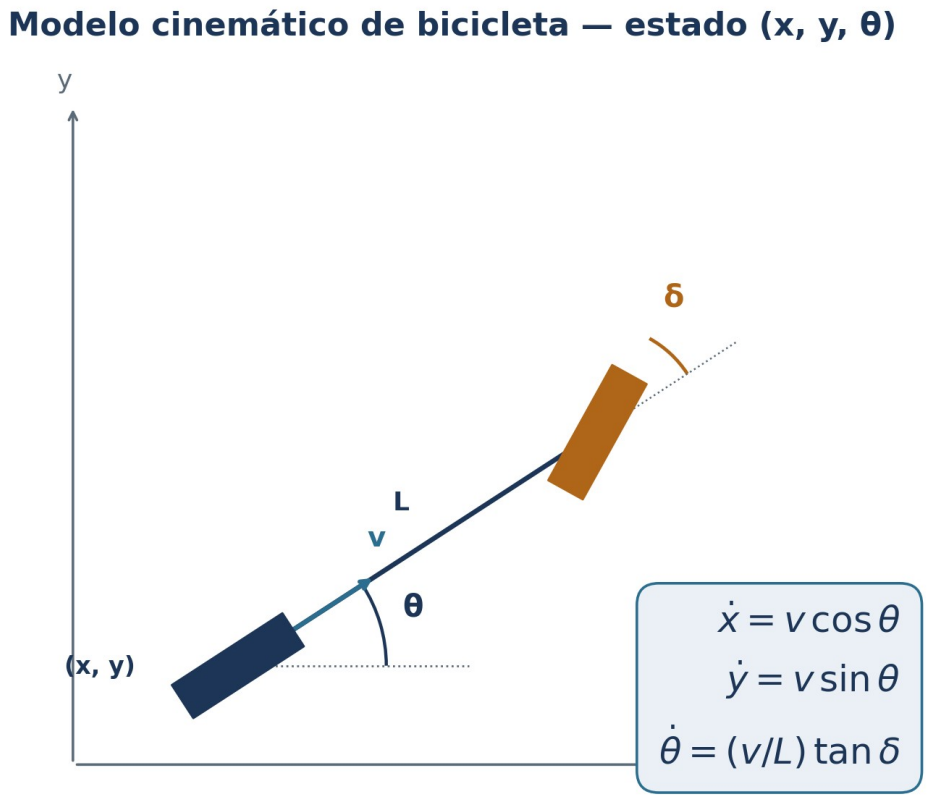
IDEA CLAVE

De la geometría al límite del planificador

El ángulo máximo de las ruedas fija el radio de giro mínimo. Reeds-Shepp sólo puede usar arcos de ese radio: la geometría restringe la planificación.

MODELADO

Modelo cinemático de bicicleta



Modelo de bicicleta: el estándar para vehículos tipo Ackermann.

- Las dos ruedas de cada eje se condensan en una sola sobre el eje longitudinal: el modelo de bicicleta.
- El estado del vehículo es la terna (x, y, θ): posición y rumbo.
- **Ecuaciones:** $\dot{x} = v\cdot\cos\theta$, $\dot{y} = v\cdot\mathrm{sen}\theta$, $\dot{\theta} = (v/L)\cdot\tan\delta$
- Las entradas son la velocidad v y el ángulo de dirección δ.
- Modelo cinemático: describe geometría y velocidades, no fuerzas. Válido con

> *Este modelo es el que usan tanto el planificador como el generador de referencias del control.*

MODELADO

Dinámica del vehículo: ecuaciones de movimiento

- El estado del auto es la terna (x, y, θ): posición del eje trasero y rumbo.
- A velocidad v sobre un círculo de radio r, la velocidad angular es ω = v / r.
- Ecuaciones de movimiento (modelo cinemático):

$$\dot{x} = \cos(\theta)$$

$$\dot{y} = \sin(\theta)$$

$$\dot{\theta} = \frac{v}{L}\tan(\delta) = \frac{v}{r_{turn}}$$

- Entradas: la velocidad v y el ángulo de dirección δ; Ackermann fija r_turn.
- Es el sistema de Dubins: 3 controles — izquierda, derecha, recto — sobre estas ecuaciones.

IDEA CLAVE

Tres ecuaciones describen todo el movimiento

Con (x, y, θ) y las entradas (v, δ) el modelo predice la pose en cada instante. Sobre él se apoyan el planificador y el generador de referencias.

Del modelo continuo al paso discreto

- El hub no integra ecuaciones: avanza el estado paso a paso, en cada tick.
- Ecuaciones de actualización, con paso Δs = v · Δt, integración de Euler:

$$x_{[k+1]} = x_{[k]} + \delta x_{[k]} = x_{[k]} + \Delta s \cos(\theta_{[k]})$$

$$y_{[k+1]} = y_{[k]} + \delta \dot{y}_{[k]} = y_{[k]} + \Delta s \sin(\theta_{[k]})$$

$$\theta_{[k+1]} = \theta_{[k]} + \delta \dot{\theta}_{[k]} = \theta_{[k]} + \frac{\Delta s}{r_{turn}}$$

- Cada paso mueve el auto sobre una recta: una aproximación lineal de un sistema no lineal.
- Cuanto más chico el paso Δs, más fina la curva (como dibujar un círculo con muchos puntos).
- Es lo que hacen el generador de referencias y el simulador: integrar estas tres líneas.

IDEA CLAVE

Integrar es sumar pasos chicos

La dinámica continua se vuelve una suma discreta. Con paso de 1 ms y v = 39 cm/s el error de discretización es despreciable.

El control visto como un sistema MIMO

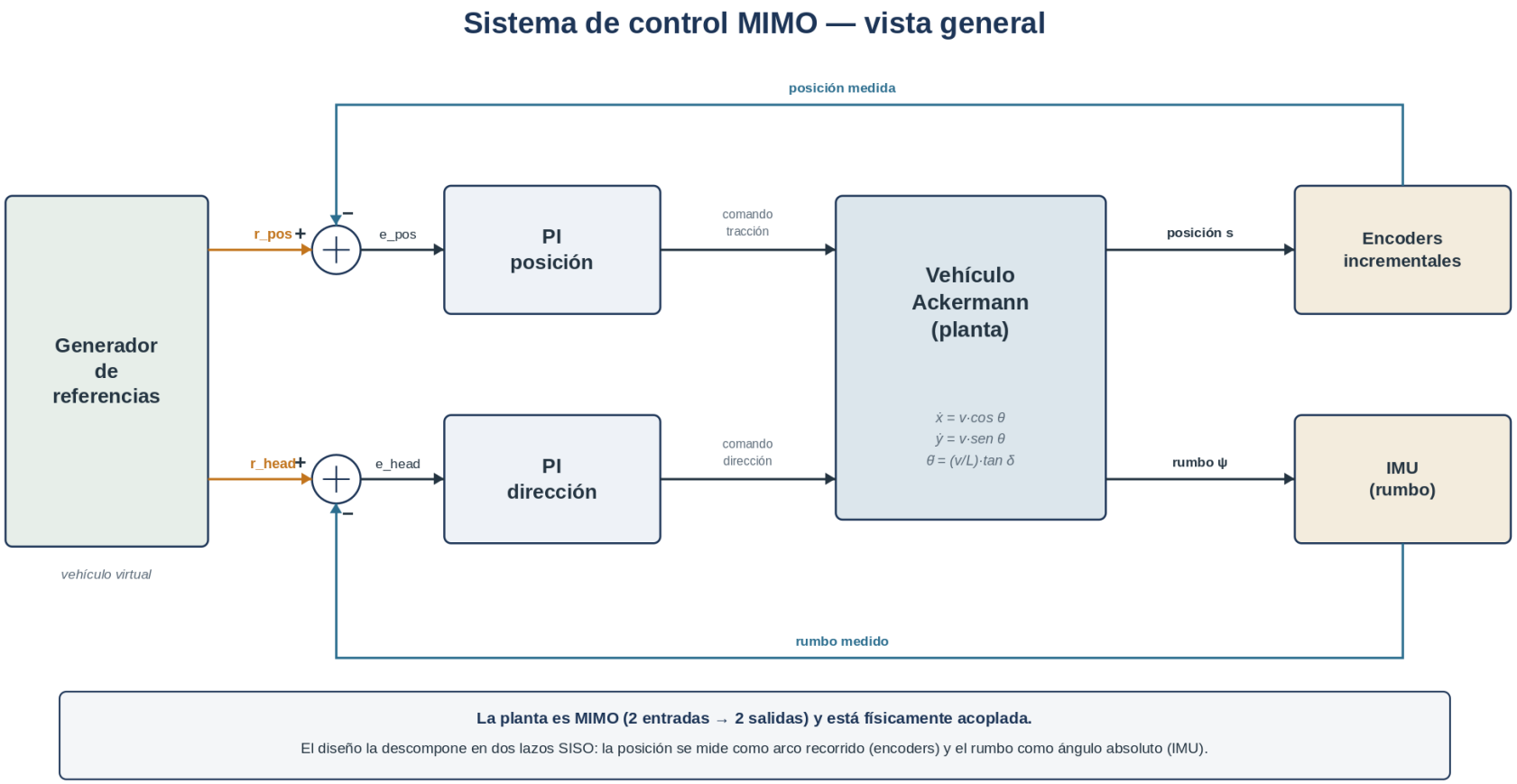
- El vehículo es una planta MIMO: múltiples entradas, múltiples salidas.
  - Entradas: comando de tracción y comando de dirección.
  - Salidas: posición recorrida y rumbo.
- Físicamente los lazos están acoplados: girar la dirección cambia el rumbo y, a la vez, curva el recorrido.
- El diseño elige medir variables que desacoplan el problema:
  - La posición se mide como arco recorrido (encoders).
  - El rumbo se mide como ángulo absoluto (IMU).
- Con esa elección, el MIMO se trata como dos lazos SISO casi independientes.

IDEA CLAVE

El acoplamiento se resuelve en la elección de variables

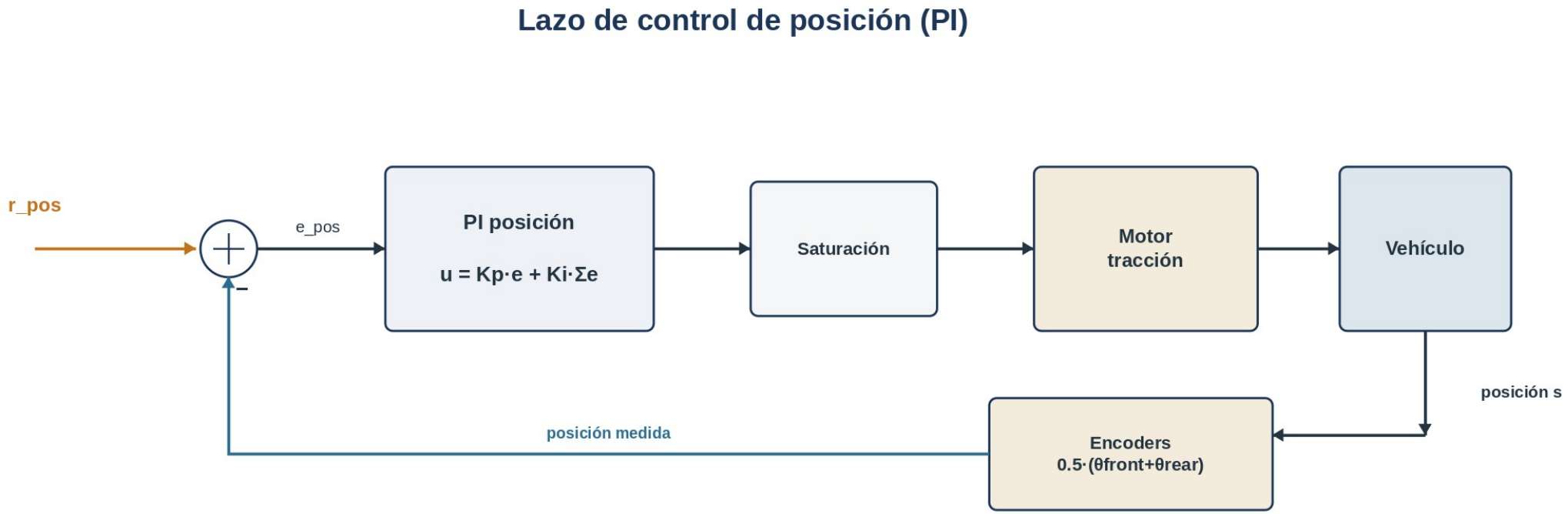
Medir el arco recorrido y el rumbo por separado convierte un problema MIMO acoplado en dos lazos simples. La geometría del giro se traslada al generador de referencias, no al control.

Diagrama de bloques MIMO



Generador de referencias → dos lazos PI → planta Ackermann → sensores → realimentación.

Lazo de control de posición



El lazo de posición regula cuánto avanzó el auto a lo largo del camino. La realimentación es el promedio de los dos encoders de tracción; su salida es la potencia del motor.

El controlador Proporcional-Integral

- **Acción proporcional (P):** $u_P = K_p \cdot e$
  - Responde al error presente. Reacciona rápido, pero por sí sola deja error permanente.
- **Acción integral (I):** $u_I = K_i \cdot \Sigma e$
  - Acumula el error pasado. Elimina el error permanente: mientras quede error, sigue creciendo.
- La suma P + I combina respuesta rápida y exactitud en régimen permanente.
- No se usa término derivativo (D): amplifica el ruido de los sensores y el sistema, lento y sobreamortiguado, no lo necesita.

IDEA CLAVE

P para reaccionar, I para no quedar corto

El proporcional da velocidad de respuesta; el integral garantiza que, en régimen, el auto efectivamente llegue a la referencia. Sin I quedaría siempre un poco atrás.

El PI en el código

```
# --- main_rs.py : dentro del lazo de control ---

# Errores
position_error = position_reference - position_vehicle
steering_error = degrees(steering_reference) - steering_vehicle

# Accion de POSICION (traccion), PI
position_P  = GAINS['kp_pos'] * position_error
position_I += GAINS['ki_pos'] * position_error
position_command = saturate(position_P + position_I,
                            max_drive_power)

# Accion de DIRECCION (steering), PI
steering_P  = GAINS['kp_st'] * steering_error
steering_I += GAINS['ki_st'] * steering_error
steering_command = gear * saturate(steering_P + steering_I,
                                   max_angle_power)
```

saturate() recorta cada comando a ±100 y lo devuelve entero.

- Cada controlador queda en tres líneas: P, I (acumulador con +=) y el comando.
- saturate() recorta a ±100 y entrega entero: una sola función para los dos lazos.
- El error de rumbo compara grados con grados (referencia convertida con degrees()).
- steering_command se multiplica por gear: en marcha atrás se invierte la corrección.
- Ganancias semilla en controlConstants.py (Kp=50, Ki=0,01); el dashboard las ajusta por ensayo.

El generador de referencias en el código

```
# main_rs.py  ·  delta_s definido una vez:
delta_s = h * v / 1000.0   # avance por tick (h=1ms, v=39cm/s)

# ... dentro del lazo, en cada iteracion: ...

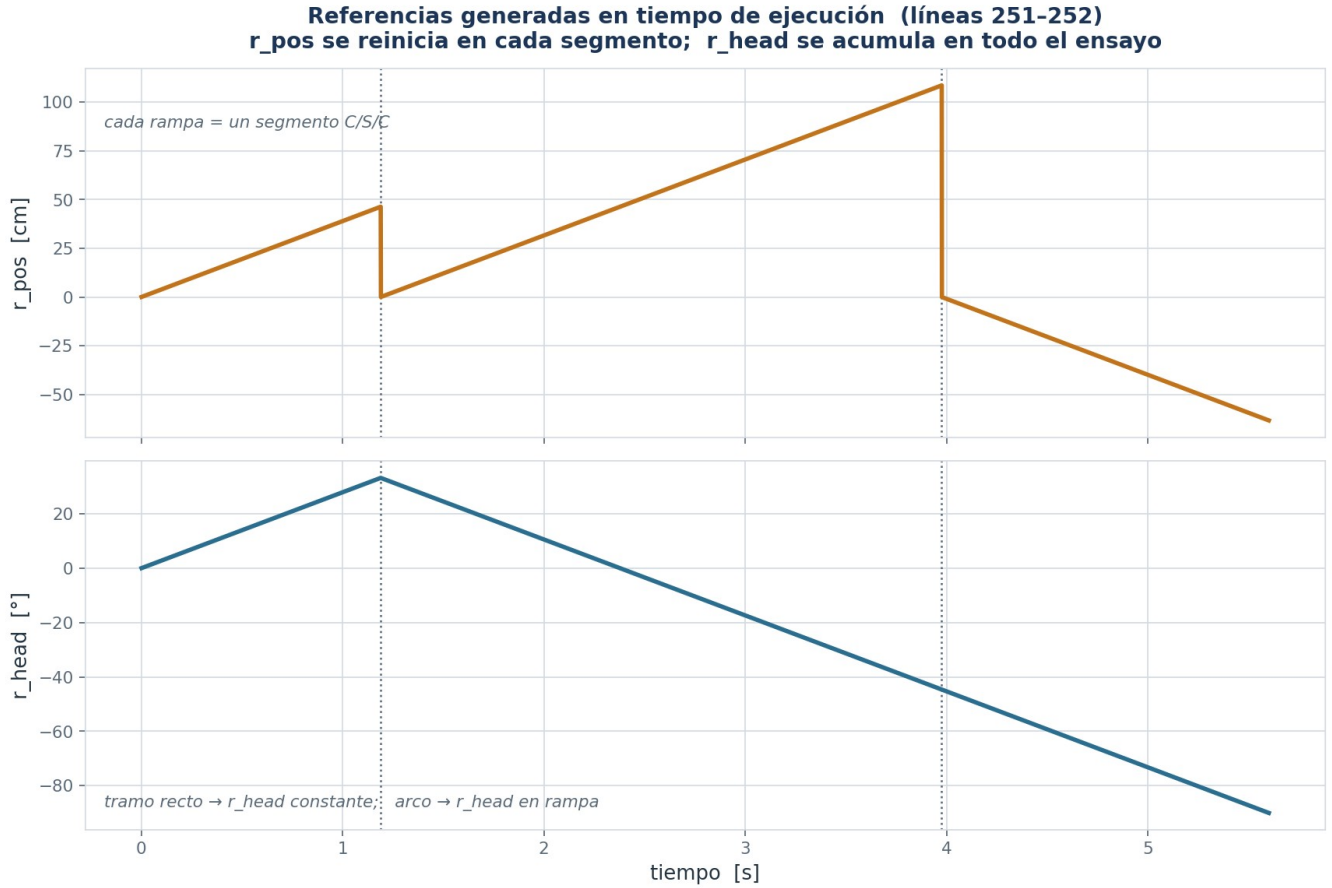
# las referencias avanzan juntas, solo mientras
# la referencia no llego al final del tramo
if abs(position_reference) < abs(distance):
    position_reference += gear * delta_s
    steering_reference += gear * steer_dir * delta_s / r_turn_min

#   r_pos  : integra el ARCO recorrido
#   r_head : integra el RUMBO  (d-theta = ds / R)
```

El generador de referencias completo: dos integradores discretos dentro del mismo lazo de control.

- Cada tick (1 ms) la referencia avanza $\Delta s = h \cdot v/1000$.
- r_pos integra distancia; gear da el signo (adelante o marcha atrás).
- r_head integra rumbo: relación cinemática $d\theta = ds/R$, con R = r_turn_min.
- En un tramo recto steer_dir = 0: el rumbo de referencia queda constante.
- Al terminar el tramo, la compuerta if … < distance deja la referencia quieta esperando al auto.

Las referencias en el tiempo



Salida del generador de referencias del vehículo virtual.

- Simulación del generador para la trayectoria de ejemplo (0,0,0°)→(120,60,−90°).
- r_pos: una rampa por segmento. Se reinicia a cero al empezar cada segmento C/S/C.
- r_head: señal continua en todo el ensayo. No se reinicia entre segmentos.
- En los tramos curvos r_head es una rampa; en los rectos sería un tramo plano.
- La pendiente de cada rampa de r_head depende del radio y del sentido de giro.

r_pos se reinicia por segmento; r_head se acumula en todo el ensayo. Por eso los encoders se ponen a cero por segmento y la IMU sólo al inicio.

SENSORES

Dos sensores, dos lazos

IMU — rumbo / heading

- Mide el rumbo absoluto del vehículo (orientación).
- Integrada en el hub LEGO Technic.
- Cierra el lazo de dirección.
- Se lee con hub.imu.heading(), en grados.
- Se pone a cero una vez, al inicio de la trayectoria.

Encoders — avance / odometría

- Miden el ángulo girado por cada motor de tracción.
- Uno por cada motor (delantero y trasero).
- Cierran el lazo de posición.
- Se leen con front.angle() y rear.angle(), en grados.
- Se ponen a cero al inicio de cada segmento.

De pulsos a centímetros: odometría

```
# main_rs.py
ANGLE_TO_CM = 0.045    # cm por grado de rueda

# al empezar cada segmento, los encoders a cero:
front.reset_angle(0)
rear.reset_angle(0)

# posicion = promedio de los dos encoders, en cm
position_vehicle = 0.5 * (front.angle() + rear.angle()) \
                       * ANGLE_TO_CM
```

Odometría: el avance del auto medido como promedio de los dos encoders.

- Pybricks ya entrega la cuenta del encoder resuelta: Motor.angle() devuelve grados acumulados.
- El factor ANGLE_TO_CM convierte grados de rueda en centímetros recorridos.
- Se promedian los dos motores de tracción: filtra pequeñas diferencias entre ruedas.
- Reset por segmento: la posición de referencia también arranca de cero en cada segmento.

Cómo se comandan los motores

```
# main_rs.py  ·  configuracion del hardware
from pybricks.robotics import Car
from pybricks.pupdevices import Motor

steering = Motor(Port.D, Direction.CLOCKWISE)
front    = Motor(Port.B, Direction.CLOCKWISE)
rear     = Motor(Port.A, Direction.CLOCKWISE)
audi     = Car(steering, [front, rear])

# ... dentro del lazo de control: ...
audi.steer(steering_command)        # -100..100 direccion
audi.drive_power(position_command)  # -100..100 traccion

if rear.stalled() or front.stalled():   # motor trabado
    stop_car()              # frena traccion y direccion
```

El control entrega dos números por ciclo; Pybricks los traduce en movimiento.

- Tres motores iguales: dos de tracción (A, B) y uno de dirección (D).
- El encoder integrado hace que cada motor de tracción sea también sensor.
- steer() posiciona las ruedas; drive_power() fija la potencia de avance.
- stalled() detecta motor bloqueado y llama a stop_car(): frena tracción y dirección de una.

Dubins y Reeds-Shepp

Curvas de Dubins

- Camino más corto entre dos poses con radio de giro mínimo.
- Sólo movimiento hacia adelante.
- El vehículo no puede dar marcha atrás.
- Útil para aviones o vehículos que no reversan.
- Es un caso particular de Reeds-Shepp.

Curvas de Reeds-Shepp

- Camino más corto admitiendo también marcha atrás.
- Aparecen los 'cusps': puntos donde se invierte el sentido.
- Mucho más flexibles para maniobrar en poco espacio.
- Es el caso general: incluye a Dubins.
- Es el que usa este proyecto.

Cómo se elige el camino óptimo

```
# reeds_shepp.py
def get_all_paths(start, end):
    # 12 familias x 4 variantes (timeflip, reflect)
    # cada funcion path1..path12 prueba una familia
    ...
    return paths        # todos los caminos validos

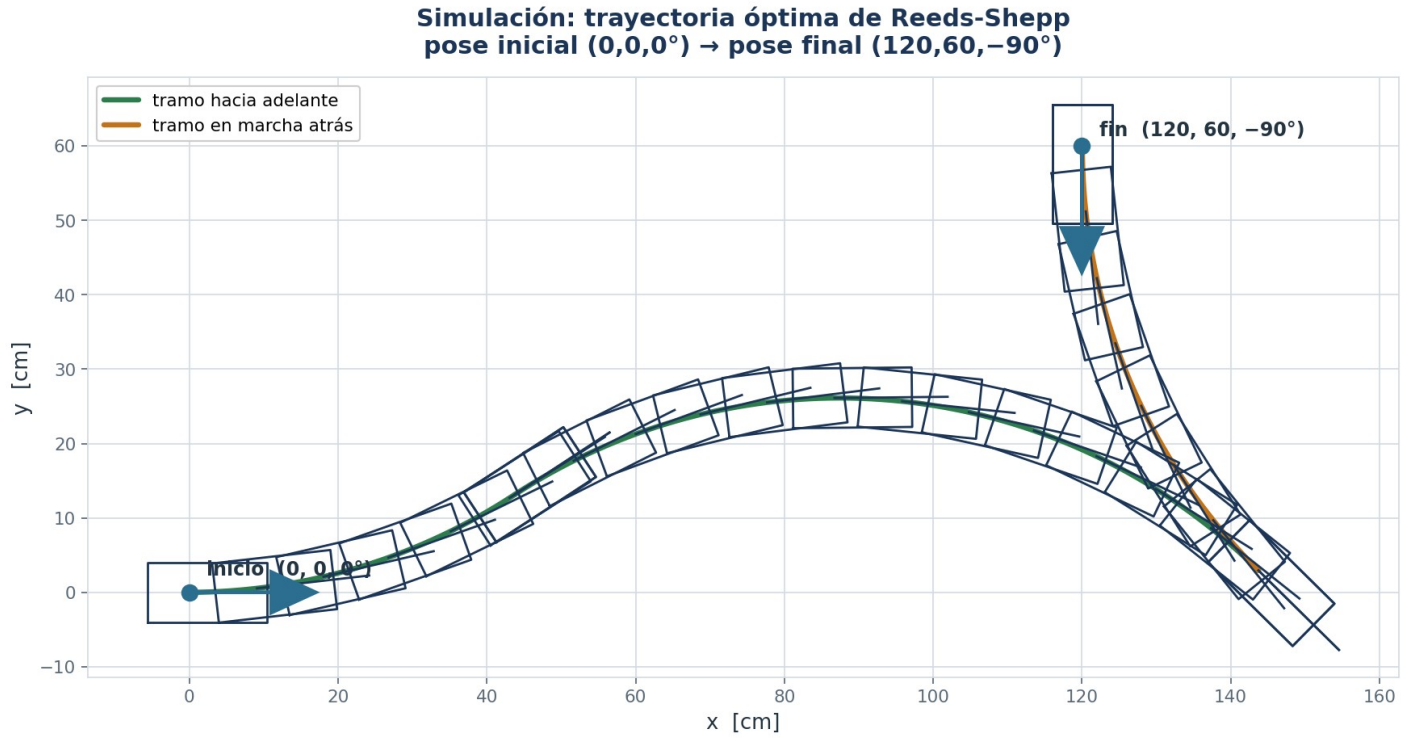
def get_optimal_path(paths):
    min_distance = infinito
    for i, path in enumerate(paths):
        # longitud total = suma de los segmentos
        d = sum(seg['distance'] for seg in path)
        if d < min_distance:
            min_distance = d; idx = i
    return idx          # el mas corto
```

reeds_shepp.py — se generan todos los caminos y se elige el más corto.

- Para cada par de poses se generan todos los caminos posibles: 12 familias por 4 variantes geométricas.
- Cada variante se obtiene reflejando o invirtiendo el tiempo (timeflip, reflect): truco para no escribir 48 fórmulas.
- El criterio de óptimo es simple: menor longitud total.
- El camino elegido se escala al radio real y se entrega al generador de referencias.

SIMULACIÓN

Resultado de la simulación



Simulación generada con reeds_shepp.py — la misma lógica que corre en el hub.

- Trayectoria óptima de Reeds-Shepp calculada con el algoritmo del proyecto.
- Pose inicial (0,0,0°), pose final (120,60,−90°).
- El camino combina tramos hacia adelante (verde) y un tramo en marcha atrás (naranja).
- Las siluetas muestran al auto y su rumbo a lo largo del recorrido.
- El auto llega exactamente a la pose final pedida: el algoritmo es correcto.

Esta figura se generó con el código real del repositorio: no es un dibujo, es la salida verificada del algoritmo.

El bucle de control en tiempo real

```
# main_rs.py  ·  controlDelVehiculo()  (esquema)
while abs(position_vehicle) <= distance and not timeout:

    if read_command() == 'STOP':      # parada de emergencia
        return 'stop'
    if rear.stalled() or front.stalled():   # motor trabado
        return 'stall'

    # 1 - avanzar las referencias   (r_pos, r_head)
    # 2 - leer sensores             (encoders + IMU)
    # 3 - calcular errores y accion PI
    # 4 - saturar y aplicar a los motores
    audi.steer(steering_command)
    audi.drive_power(position_command)

    wait(h)                          # h = 1 ms
    elapsed_time += h
```

controlDelVehiculo() — un segmento C/S/C por llamada.

- Un único lazo, period fijo de 1 ms, repite cuatro pasos: referencia, medición, control, actuación.
- El lazo termina cuando el auto recorrió la distancia del segmento (o por watchdog de tiempo).
- Las verificaciones de STOP y de motor trabado están al principio: la seguridad se chequea antes que nada.
- Es la estructura clásica de un lazo de control embebido.

IMPLEMENTACIÓN

Seguridad y robustez

- Watchdog de tiempo: cada segmento tiene un tiempo máximo (30 s); si se excede, el ensayo termina con 'timeout'.
- Detección de motor bloqueado: stalled() corta tracción y dirección si el auto se traba.
- Parada de emergencia: el comando STOP del dashboard se atiende dentro del lazo, en cada iteración.
- Gestión de memoria: gc.collect() libera memoria entre tramos — el hub tiene RAM muy limitada.
- El planificador puede no encontrar solución: ese caso se detecta y se informa, no se ignora.
- Cada ensayo termina siempre con un estado explícito: ok, stop, stall, timeout o error.

IDEA CLAVE

Un sistema embebido tiene que fallar de forma segura

No alcanza con que ande cuando todo sale bien. Watchdog, detección de bloqueo y parada de emergencia definen qué pasa cuando algo sale mal.

VALIDACIÓN

¿Cómo sabemos que el sistema funciona?

- Criterio visual: en el dashboard, la trayectoria real debe superponerse a la planificada.
- Criterio cuantitativo: error RMS de posición y de rumbo durante todo el ensayo.
- Error máximo: el peor apartamiento respecto de la referencia — importante cerca de los cusps.
- Estado final del ensayo: debe terminar en 'ok', no en stall ni timeout.
- Pose final: el auto debe quedar en la pose pedida, dentro de una tolerancia.
- Comparación contra la simulación: la trayectoria real no debería alejarse de la simulada.

IDEA CLAVE

Medir, no opinar

'Anda bien' no es un criterio. RMS, error máximo y estado final sí lo son. El dashboard los calcula automáticamente al cerrar cada ensayo.

CIERRE

Limitaciones y mejoras posibles

- Anti-windup explícito: congelar el integral mientras el comando está saturado.
- Velocidad constante: hoy v es fija; desacelerar cerca de los cusps mejoraría la precisión.
- Deriva de la IMU: en recorridos largos convendría corregir el rumbo.
- Odometría sin corrección: los encoders no detectan deslizamiento de las ruedas.
- Convención de signo: el simulador MATLAB y el código del hub usan signos opuestos en la integración del rumbo — conviene unificar y documentar.
- Sin evitación de obstáculos: el planificador asume el espacio libre.

IDEA CLAVE

Conocer los límites es parte del diseño

Ninguna de estas limitaciones impide que el sistema funcione. Pero tenerlas identificadas es lo que permite mejorarlo con criterio.

CIERRE

Referencias y material del proyecto

- Reeds, J. A. & Shepp, L. A. (1990). Optimal paths for a car that goes both forwards and backwards. Pacific Journal of Mathematics, 145(2).
- Dubins, L. E. (1957). On curves of minimal length with a constraint on average curvature. American Journal of Mathematics.
- Implementación de referencia de Reeds-Shepp: repositorio de N. Lecointe (github.com/nathanlct/reeds-shepp-curves).
- Pedroni, J. P. Práctico de Laboratorio N°2 — Decodificación de encoder incremental (material de cátedra).
- Documentación de Pybricks: pybricks.com — API de motores, IMU y robótica.
- Geometría de Ackermann: artículo de Wikipedia 'Ackermann steering geometry'.
- Código del proyecto: main_rs.py, reeds_shepp.py, dashboard.html y la carpeta Matlab/ del repositorio.
- Giese, A. (2012). A Comprehensive, Step-by-Step Tutorial on Computing Dubin's Curves.
- Teodovich Sosa, L. J. & Carelli, R. (2008). Control Híbrido para Posicionamiento de un Robot tipo Ackerman. JAR'08, UNS.
- Ackermann Chassis Kinematics — nota técnica: ángulos de dirección y velocidades por rueda.